Assignment - 9.4

M.Koushik Reddy
2103a52153

Task 1

Auto-Generating Function Documentation in a Shared Codebase

```
def calculate_area(length, width):
    """
    Calculates the area of a rectangle.

    Args:
        length (float or int): The length of the rectangle.
        width (float or int): The width of the rectangle.

    Returns:
        float: The calculated area of the rectangle.

    Example:
        >>> calculate_area(5, 3)
        15
    """
    return length * width


def is_even(number):
    """
    Determines whether a given number is even.
```

```
    Args:
        number (int): The number to check.

    Returns:
        bool: True if the number is even, False otherwise.

    Example:
        >>> is_even(4)
        True
        >>> is_even(7)
        False
    """
    return number % 2 == 0


def find_max(numbers):
    """
    Finds the maximum value in a list of numbers.

    Args:
        numbers (list of int or float): A list containing numeric values.

    Returns:
        int or float: The largest number in the list.

    Raises:
        ValueError: If the list is empty.

    Example:
        >>> find_max([1, 5, 3, 9, 2])
        9
    """
    if not numbers:
        raise ValueError("The numbers list cannot be empty.")
    return max(numbers)


def greet_user(name, greeting="Hello"):
    """
    Generates a greeting message for a user.
```

```
    Args:
        name (str): The name of the user.
        greeting (str, optional): The greeting phrase. Defaults to
"Hello".

    Returns:
        str: A personalized greeting message.

    Example:
        >>> greet_user("Alice")
        'Hello, Alice!'
        >>> greet_user("Bob", "Good morning")
        'Good morning, Bob!'
    """
    return f"{greeting}, {name}!"


# ---- Program Execution ----
if __name__ == "__main__":
    print("Area:", calculate_area(5, 3))
    print("Is 4 even?", is_even(4))
    print("Is 7 even?", is_even(7))
    print("Maximum number:", find_max([1, 5, 3, 9, 2]))
    print(greet_user("Alice"))
    print(greet_user("Bob", "Good morning"))
```

Output

```
PS C:\Users\koushik reddy\Desktop\python practice> & "C:/Users
Area: 15
Is 4 even? True
Is 7 even? False
Maximum number: 9
Hello, Alice!
Good morning, Bob!
PS C:\Users\koushik reddy\Desktop\python practice>
```

Task 2 - Enhancing Readability Through AI-Generated Inline Comments

```
def fibonacci(n):
    sequence = [0, 1]

    for i in range(2, n):
        # Each new number is calculated using the sum of the previous two numbers,
        # which avoids recursion and improves performance for larger n
        next_value = sequence[i - 1] + sequence[i - 2]
        sequence.append(next_value)

    return sequence[:n]


def custom_sort(numbers):
    # Using Bubble Sort for demonstration of algorithmic logic
    length = len(numbers)

    for i in range(length):
        swapped = False  # Tracks if any swap occurs to optimize unnecessary passes

        for j in range(0, length - i - 1):
            # Compare adjacent elements and swap if they are in wrong order
            if numbers[j] > numbers[j + 1]:
```

```
                numbers[j], numbers[j + 1] = numbers[j + 1], numbers[j]
                swapped = True

        # If no swaps happened in this pass, the list is already sorted
        if not swapped:
            break

    return numbers


def search_and_process(data, target):
    results = []

    for value in data:
        # Skip negative numbers because the system only processes valid
positive entries
        if value < 0:
            continue

        # Stop processing entirely if target is found,
        # assuming data beyond this point is irrelevant
        if value == target:
            break

        # Store squares of values divisible by 2 or 3,
        # representing a custom filtering rule
        if value % 2 == 0 or value % 3 == 0:
            results.append(value ** 2)

    return results


if __name__ == "__main__":
    fib_numbers = fibonacci(10)
    print("Fibonacci:", fib_numbers)

    unsorted_numbers = [64, 34, 25, 12, 22, 11, 90]
    sorted_numbers = custom_sort(unsorted_numbers)
    print("Sorted:", sorted_numbers)
```

```
    processed = search_and_process([5, 8, -3, 12, 7, 9, 15], 9)
    print("Processed:", processed)
```

Output

```
Fibonacci: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
Sorted: [11, 12, 22, 25, 34, 64, 90]
Processed: [64, 144]
PS C:\Users\koushik reddy\Desktop\python practice>
```

Task 3 - Generating Module-Level Documentation for a Python Package

```
data_processing.py

Purpose:
This module provides utility functions and classes for processing
numerical
datasets. It supports generating Fibonacci sequences, sorting numerical
data,
and filtering datasets based on custom rules. The module is designed to
help
developers perform common data transformation tasks efficiently.

Dependencies:
- Python Standard Library only (no external packages required)

Key Components:
- fibonacci(n):
```

```
    Generates the Fibonacci sequence up to n elements using an iterative
approach.

- DataSorter:
    A class that implements a bubble sort algorithm with an optimization
that
    stops execution early if the dataset is already sorted.

- filter_and_transform(data, stop_value):
    Filters out invalid data and transforms values based on defined rules.

Example Usage:
    from data_processing import fibonacci, DataSorter,
filter_and_transform

    # Generate Fibonacci sequence
    fib_numbers = fibonacci(8)
    print(fib_numbers)

    # Sort numbers using DataSorter
    sorter = DataSorter([5, 3, 8, 1])
    sorted_numbers = sorter.sort()
    print(sorted_numbers)

    # Filter and transform dataset
    processed = filter_and_transform([4, -2, 6, 7, 9], 7)
    print(processed)
"""


def fibonacci(n):
    """Generate Fibonacci sequence up to n elements."""
    sequence = [0, 1]

    for i in range(2, n):
        next_value = sequence[i - 1] + sequence[i - 2]
        sequence.append(next_value)

    return sequence[:n]
```

```
class DataSorter:
    """Sorts numerical data using bubble sort."""

    def __init__(self, numbers):
        self.numbers = numbers

    def sort(self):
        length = len(self.numbers)

        for i in range(length):
            swapped = False

            for j in range(0, length - i - 1):
                if self.numbers[j] > self.numbers[j + 1]:
                    self.numbers[j], self.numbers[j + 1] = (
                        self.numbers[j + 1],
                        self.numbers[j],
                    )
                    swapped = True

            if not swapped:
                break

        return self.numbers


def filter_and_transform(data, stop_value):
    """
    Filters negative numbers and transforms selected values.
    Stops processing when stop_value is encountered.
    """
    results = []

    for value in data:
        if value < 0:
            continue

        if value == stop_value:
            break
```

```
        if value % 2 == 0 or value % 3 == 0:
            results.append(value ** 2)

    return results


if __name__ == "__main__":
    print("Fibonacci:", fibonacci(10))

    sorter = DataSorter([64, 34, 25, 12, 22, 11, 90])
    print("Sorted:", sorter.sort())

    print("Processed:", filter_and_transform([5, 8, -3, 12, 7, 9, 15], 9))
```

Output

```
Fibonacci: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
Sorted: [11, 12, 22, 25, 34, 64, 90]
Processed: [64, 144]
```

Task 4: Converting Developer Comments into Structured Docstrings

Before

```
def calculate_average(numbers):
    # This function takes a list of numbers and calculates their average.
    # First, it checks if the list is empty because dividing by zero
    # would cause an error in the program.
    if len(numbers) == 0:
        return 0

    # The total variable stores the sum of all numbers in the list.
```

```
    total = 0

    # Loop through each number in the list and add it to total.
    for num in numbers:
        total += num

    # Divide the total sum by the number of elements in the list
    # to calculate the average value.
    average = total / len(numbers)

    return average


def find_maximum(numbers):
    # This function finds the maximum number in a list.
    # It assumes the first element is the largest initially.
    if not numbers:
        return None

    max_value = numbers[0]

    # Loop through remaining elements to compare values.
    for num in numbers[1:]:
        # If current number is greater than max_value,
        # update max_value.
        if num > max_value:
            max_value = num

    return max_value
```

After

```
def calculate_average(numbers):
    """
    Calculate the average value of a list of numbers.
```

```
    Args:
        numbers (list of float or int): A list containing numerical
values.

    Returns:
        float: The average of the numbers in the list. Returns 0 if the
list is empty.
    """
    if len(numbers) == 0:
        return 0

    total = 0
    for num in numbers:
        total += num

    return total / len(numbers)


def find_maximum(numbers):
    """
    Find the maximum value in a list of numbers.

    Args:
        numbers (list of float or int): A list containing numerical
values.

    Returns:
        float or int or None: The maximum value in the list.
        Returns None if the list is empty.
    """
    if not numbers:
        return None

    max_value = numbers[0]

    for num in numbers[1:]:
        if num > max_value:
            max_value = num

    return max_value
```

Task 5: Building a Mini Automatic Documentation Generator

Python Documentation Scaffolding Utility

```
import ast


def generate_function_docstring(function_name, args):
    """Generate placeholder Google-style docstring for functions."""
    params = "\n".join(
        [f"        {arg} (TYPE):: Add description." for arg in args]
    ) or "        None"

    return f'''    """
    TODO: Describe the purpose of {function_name}.

    Args:
{params}

    Returns:
        TYPE: Describe return value.
    """'''


def generate_class_docstring(class_name):
    """Generate placeholder Google-style docstring for classes."""
    return f'''    """
    TODO: Describe the purpose of the {class_name} class.

    Attributes:
        TODO: Add class attributes.
```

```
    Methods:
        TODO: Describe important methods.
    """'''


class DocstringInserter(ast.NodeTransformer):
    """AST transformer that inserts placeholder docstrings."""

    def visit_FunctionDef(self, node):
        # Skip if docstring already exists
        if ast.get_docstring(node) is None:
            args = [arg.arg for arg in node.args.args]

            docstring = generate_function_docstring(node.name, args)

            node.body.insert(
                0,
                ast.Expr(value=ast.Constant(value=docstring.strip()))
            )

        self.generic_visit(node)
        return node

    def visit_ClassDef(self, node):
        # Skip if docstring already exists
        if ast.get_docstring(node) is None:
            docstring = generate_class_docstring(node.name)

            node.body.insert(
                0,
                ast.Expr(value=ast.Constant(value=docstring.strip()))
            )

        self.generic_visit(node)
        return node


def insert_docstrings(input_file, output_file):
    """
    Read a Python file, insert placeholder docstrings, and save output.
```

```
    """
    with open(input_file, "r", encoding="utf-8") as file:
        source_code = file.read()

    tree = ast.parse(source_code)

    transformer = DocstringInserter()
    updated_tree = transformer.visit(tree)

    updated_code = ast.unparse(updated_tree)

    with open(output_file, "w", encoding="utf-8") as file:
        file.write(updated_code)

    print(f"Docstrings inserted into {output_file}")


if __name__ == "__main__":
    input_file = "input_script.py"    # File to process
    output_file = "output_script.py" # File with inserted docstrings

    insert_docstrings(input_file, output_file)
```

Example Output File (output_script.py)

```
class Calculator:
    """
    TODO: Describe the purpose of the Calculator class.

    Attributes:
        TODO: Add class attributes.

    Methods:
        TODO: Describe important methods.
    """

    def add(self, a, b):
```

```
        """
        TODO: Describe the purpose of add.

        Args:
            a (TYPE): Add description.
            b (TYPE): Add description.

        Returns:
            TYPE: Describe return value.
        """
        return a + b


def multiply(x, y):
    """
    TODO: Describe the purpose of multiply.

    Args:
        x (TYPE): Add description.
        y (TYPE): Add description.

    Returns:
        TYPE: Describe return value.
    """
    return x * y
```